

Drop Ceiling — Behavior System Diagrams

A visual walkthrough of how the Drop Ceiling light thinks, moves, and learns — from camera input to physical light output.

Each diagram builds on the previous one, starting with the highest-level overview and drilling progressively deeper into the system's internals. If you're new to the project, read them in order.

All diagrams use [Mermaid](#) syntax and render natively on GitHub.

1. System Overview

The Drop Ceiling installation is a single simulated point light that moves above a grid of LED panels. Two cameras watch pedestrians below, and the light responds in real-time.

At the highest level, data flows through six stages: two RTSP cameras capture video, YOLO detects people, calibration projects detections to real-world floor coordinates, cross-camera fusion and temporal tracking produce stable positions, the behavior system decides what the light should do, and Art-Net sends DMX values to the physical panels.



Three Python files implement the system:

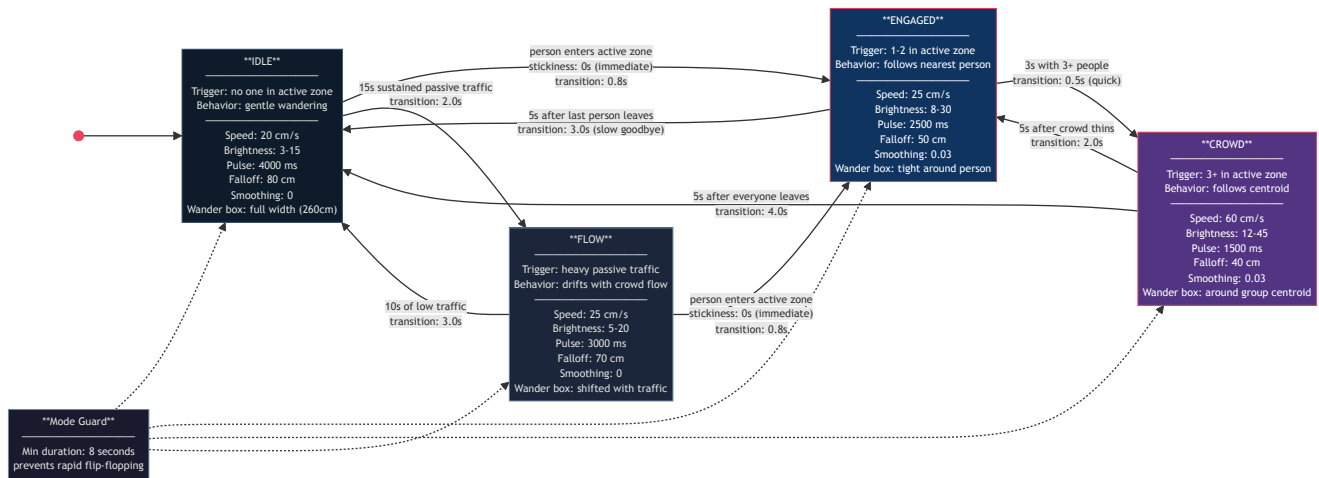
File	Responsibility
<code>camera_tracker_osc.py</code>	Camera capture, YOLO detection, calibration, fusion, temporal tracking, OSC output
<code>light_behavior.py</code>	State machine, gestures, trend analysis, feedback learning
<code>lightController_osc.py</code>	Main loop, OSC input, point light, panel math, Art-Net output, auto-tuning

The behavior system outputs a **params dict** every frame containing target values for brightness, speed, falloff radius, pulse rate, follow smoothing, and wander interval. The controller interpolates toward these targets and computes per-panel DMX values.

2. Behavior Mode State Machine

The light is always in one of four modes. Mode determines the base personality — how fast it moves, how bright it shines, and whether it follows someone or wanders on its own.

Transitions are **not instantaneous**. Conditions must persist for a minimum duration (stickiness) before the mode switches, and parameters interpolate smoothly over a transition period. This prevents erratic flickering between states.



Key design choice: Engaging is fast (0.5–0.8s), disengaging is slow (2.0–4.0s). The light is eager to connect and reluctant to let go.

3. The 17-Layer Parameter Pipeline

This is the heart of the system. Every frame, `calculate_parameters()` in `light_behavior.py` builds the output params dict by passing it through 17 sequential layers. Each layer can modify brightness, speed, falloff, pulse rate, smoothing, and/or wander interval.

The layers are grouped into four stages: **Foundation** sets the starting point, **Personality & Context** applies the light's character and time awareness, **Environmental Response** reacts to what's happening around the light, and **Overlays** add momentary expressive effects.

1 - Foundation

Mode Base Values

Input: current BehaviorMode
Lookup: MODE_PARAMS table
Sets: all 7 output params



Transition Interpolation

Condition: mode is switching
Method: lerp old params to new
Duration: 0.5 - 4.0s per type



People-Count Scaling

Input: active person count
Effect: +20% brightness per person
Affects: brightness_min, brightness_max



2 - Personality and Context

MetaParameter Modifiers

Input: 6 personality sliders (0-1)
Input: 6 global multipliers (default 1.0)

Method: lerp + multiply
Affects: speed, brightness, pulse, smoothing, wander

Time-of-Day

Input: current hour
Brightness: x0.4 (night) to x1.1 (rush)
Pulse: x0.7 to x1.5
Wander Y: constrained by period

Dwell Rewards

Input: person dwell time
Phases: Notice / Greet / Engage / Bond
Effect: longer dwell = brighter, tighter
Scale: dwell_influence multiplier

Anti-Repetition

Input: recent gesture + position history
Scale: memory meta parameter
Effect: suppresses repeated patterns

3 - Environmental Response

Idle Trends

Windows: 1m / 5m / 30m / 1h

Derived: anticipation (0-1)

Derived: energy_level (0-1)

Derived: flow_momentum (-1 to +1)



Aggression

Input: time without engagement

Range: 0-1 (EMA smoothed)

Cap: time-of-day dependent

Effect: wander width, pulse, gesture rate



Flow Positioning

Input: flow direction + strength

Effect: wander box X shifts toward
incoming pedestrian traffic



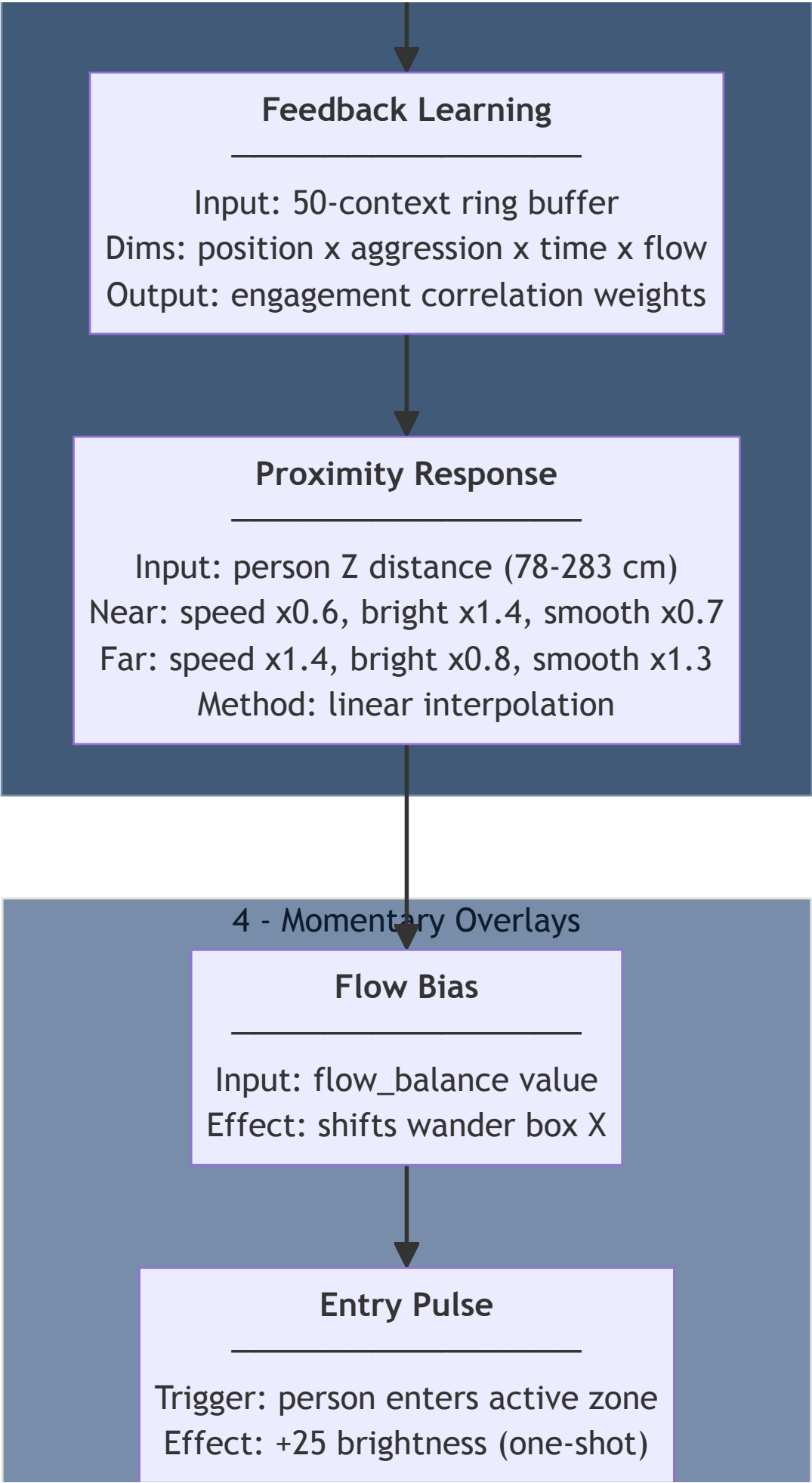
Almost-Engaged Attraction

Input: people slowing near active zone

Strategies: brightness / drift / pause

A/B tested: conversion per strategy





```
graph TD; A[Feedback Learning] --> B[Proximity Response]; B --> C[4 - Momentary Overlays]; C --> D[Flow Bias]; D --> E[Entry Pulse];
```

Feedback Learning

Input: 50-context ring buffer
Dims: position x aggression x time x flow
Output: engagement correlation weights

Proximity Response

Input: person Z distance (78-283 cm)
Near: speed x0.6, bright x1.4, smooth x0.7
Far: speed x1.4, bright x0.8, smooth x1.3
Method: linear interpolation

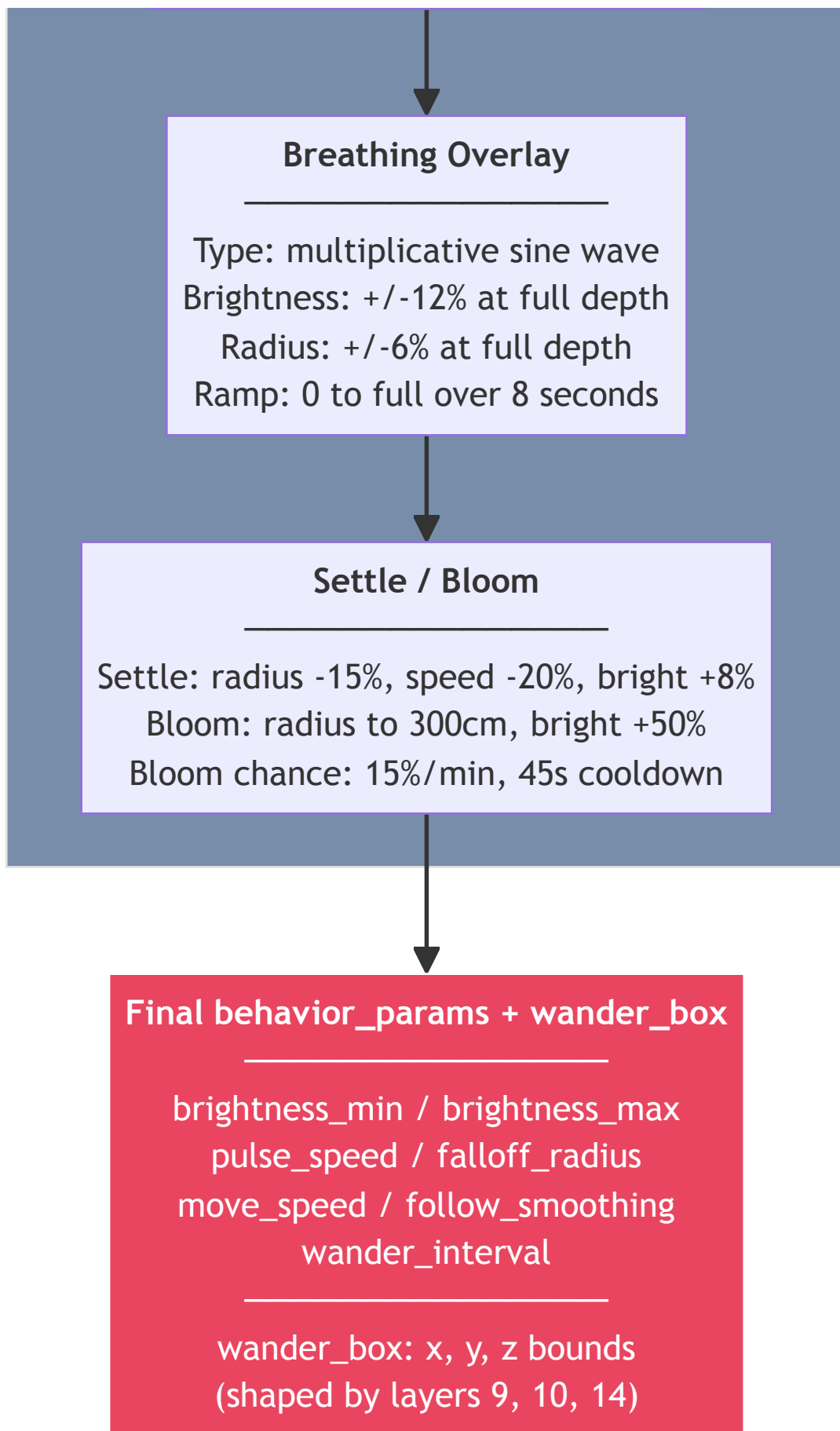
4 - Momentary Overlays

Flow Bias

Input: flow_balance value
Effect: shifts wander box X

Entry Pulse

Trigger: person enters active zone
Effect: +25 brightness (one-shot)



What each output parameter controls:

Parameter	What it does
<code>brightness_min / brightness_max</code>	DMX brightness range for the pulse cycle
<code>pulse_speed</code>	Period of the brightness sine wave (ms)
<code>falloff_radius</code>	How far from the light panels still illuminate (cm)
<code>move_speed</code>	How fast the light moves toward its target (cm/s)
<code>follow_smoothing</code>	How tightly the light tracks a person (0 = no follow, 0.2 = tight)
<code>wander_interval</code>	Time between picking new wander targets (s)
<code>wander_box</code>	3D bounding volume (x/y/z min-max) constraining light position — see Wander Box diagram

4. MetaParameters → Actual Light Values

The personality system consists of **6 sliders** (0.0–1.0) that define the light's character and **6 global multipliers** that scale the output. Together they transform the mode base values into the light's actual behavior.

`apply_meta_modifiers()` in `light_behavior.py` maps each personality slider to one or more output parameters using linear interpolation, then applies the global multipliers.

Global Multipliers (default 1.0)

trend_weight
range: 0.0 - 3.0

speed_global
range: 0.3 - 3.0

follow_speed_global
range: 0.3 - 3.0

pulse_global
range: 0.3 - 3.0

dwell_influence
range: 0.0 - 3.0

brightness_global
range: 0.3 - 5.0

Personality Sliders (0.0 - 1.0)

responsiveness
Low: contemplative, slow
High: reactive, quick

Output Parameters

move_speed
Range: x0.6 - x1.4
Unit: cm/s

follow_smoothing
Range: 0.03 - 0.20
0 = no follow

pulse_speed
Range: x1.3 - x0.7
Unit: ms period

brightness
Range: x0.7 - x1.3
Unit: DMX (1-255)

scales trend
response layer

x

lerp

x

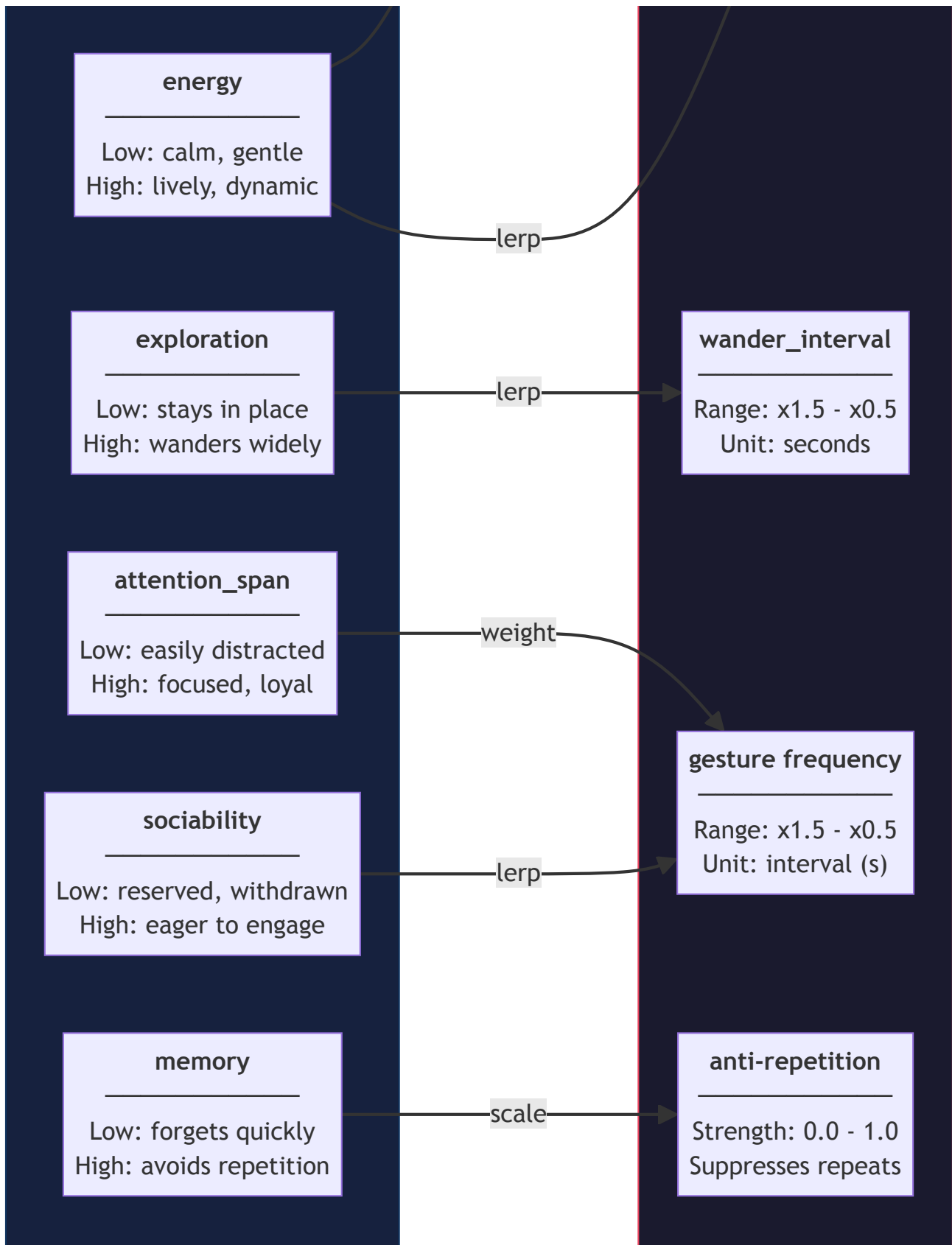
lerp

x

lerp

scales dwell
bonus layer

x



Example: With `responsiveness = 0.8` and `speed_global = 1.2`:

- Base mode speed is 20 cm/s (IDLE)
- Responsiveness maps to x1.24 (lerp between 0.6 and 1.4 at 0.8)
- Global multiplier applies: $20 \times 1.24 \times 1.2 = 29.8 \text{ cm/s}$

The sliders interact with each other through the gesture system: `sociability` controls how often gestures fire, `attention_span` influences which gestures are chosen (high attention → more SETTLE and focused gestures), and `memory` suppresses recently-used gestures.

5. AutoTuning Feedback Loop

The `AutoTuningManager` runs every 5 seconds and continuously adjusts the personality parameters based on observed activity. This creates a light that learns and adapts its character over hours and days.

The key insight is **asymmetric adjustment**: personality sliders (responsiveness, energy, sociability) are only pushed *up* when activity is high — they're never pushed down. Only mean reversion gently brings them back toward home values when things are quiet. This prevents the light from becoming permanently suppressed on a quiet sidewalk.

External Meta-Tuner

autotune_overrides.json

Hot-reloaded: every 30 seconds
Can override: home values, safe floors,
caps, curiosity, budget

override tuning
hyperparameters

Compute

Adaptive Target

Method: rolling median
Samples: ~500 (~42 min)
Clamp: 0.03 - 0.40
Purpose: relative busy/quiet

Activity Excess

Formula: short_activity
minus adaptive_target
Positive = busier than normal
Negative = quieter than normal

Sense (every 5s)

Read Aggression State

level: 0-1 (EMA smoothed)
seconds_since_engagement: int

Calculate Deltas

Exploration

When quiet: increase (search more)
When busy: decrease (stay focused)
Max step: 0.03 per cycle

Display (inverse)

Params: brightness, speed, pulse globals
When busy: decrease (personality handles it)
When quiet: increase (compensates)
Max step: 0.08 per cycle

Personality (up only)

Params: responsiveness, energy, sociability
When busy: pushed UP
When quiet: NOT pushed down
Max step: 0.03 per cycle

Adjust and Constrain

Mean Reversion

Target: home values (defaults)
Strength: $0.02 + 0.06 \times \text{distance}$
Type: progressive (stronger when far)
Always active

Curiosity Perturbation

Interval: every 30 seconds
Strength: 0.015
Bias: 60% toward home values
Purpose: explore parameter space

Budget Gate

Cost: $\text{sum}(\text{abs}(\text{deltas})) \times 60$
Restore: over ~300 seconds
Effect: scales down changes when depleted
Purpose: prevents runaway drift

Clamp

Safe floors: prevent zombie light
Soft caps: prevent obnoxious behavior
Hard range: per-parameter min/max
Min step: 0.002 (below = zeroed)

learned home values

parameter journeys
logged all day

Daily Learning (midnight)

End-of-Day Snapshot

60% final value
40% midpoint of range

Persist

Stored per time-of-day
period in database

Next Startup

Load learned values

Apply

Target: MetaParameters
Also: sync slider UI positions

MetaParameters

6 personality sliders
6 global multipliers
Updated for next frame

personality shapes
behavior output

Read Activity Levels

short_activity: 5 min window
seconds_since_engagement: int

Blend: 30% toward learned

medium_activity: 30 min window
long_activity: 1 hour window

Tuned parameters and their constraints:

Parameter	Min	Max	Safe Floor	Soft Cap	Home
responsiveness	0.1	0.95	0.30	—	0.50
energy	0.1	0.95	0.25	—	0.50
attention_span	0.1	0.95	—	—	0.50
sociability	0.1	0.95	0.20	—	0.50
exploration	0.1	0.95	—	—	0.50
memory	0.1	0.95	—	—	0.50
brightness_global	0.3	5.0	0.50	3.0	1.00
speed_global	0.3	3.0	0.40	1.6	1.00
pulse_global	0.3	3.0	—	2.0	1.00
follow_speed_global	0.3	3.0	—	—	1.00
dwel_influence	0.0	3.0	—	—	1.00
idle_trend_weight	0.0	3.0	—	—	1.00

Step sizes: Personality params max ± 0.03 per 5s cycle. Global multipliers max ± 0.08 . Changes below 0.002 are zeroed to prevent micro-drift.

6. Multi-Timescale Adaptation

The system processes information across five timescales simultaneously. Fast loops handle moment-to-moment responsiveness, while slow loops gradually shape the light's long-term character. This creates behavior that feels both reactive and intentional.

Per-Frame (~33ms)

Mode Switching

Check: active zone occupancy
Apply: stickiness timers

Gesture Triggering

Check: cooldowns, phase gates
Fire: weighted random selection

Proximity Response

Input: person Z distance
Scale: speed, brightness, smoothing

Position Interpolation

Method: exponential decay toward target

Rate: move_speed cm/s

Breathing Overlay

Advance: sine wave phase

Apply: brightness +/-12%, radius +/-6%

Panel Brightness

Calc: 12x distance falloff

Output: DMX values to Art-Net



Every 1.5 - 5 seconds

Flow Tracking

Interval: 1.5 seconds

Window: 30 seconds sliding

Output: direction (-1 to +1), strength (0-1)

Output: direction (0 to 1), strength (0 to 1)

Auto-Tuning Cycle

Interval: 5 seconds

Read: trends, aggression

Write: MetaParameters (12 params)

Attraction Strategy

Rotate: brightness / drift / pause

Track: conversion per strategy

Cooldown: 5 seconds



Rolling Windows (1m - 60m)

Recent (1 min)

Use: immediate reactivity

Drives: ready-for-action posture

Drives: ready for action posture

Short (5 min)

Output: short_activity weight
Feeds: auto-tuning primary signal

Medium (30 min)

Output: medium_activity weight
Use: general activity level

Long (1 hour)

Output: long_activity weight
Use: big-picture energy level

Aggression EMA

Rises: without engagement
Capped: by time-of-day table
Range: 0 - 0.8 depending on hour



Daily

Time-of-Day Modifiers

Periods: late_night / waking /
active / rush / evening
Scales: brightness, pulse, wander Y

Parameter Resets

Times: midnight / 6am / noon / 6pm
Method: 40% blend toward home values

Daily Report

Time: 12:01 AM

Content: engagement stats, population,
parameter journey summary

Daily Learning

Compute: optimal starting values
Granularity: per time-of-day period



Weekly

Engagement History

Method: 7-day weighted average
Granularity: by time period

Cap Loosening

Trigger: param consistently hits ceiling

trigger. param consistently hits ceiling
Effect: nudge cap up 10% for next day

Feedback Learning

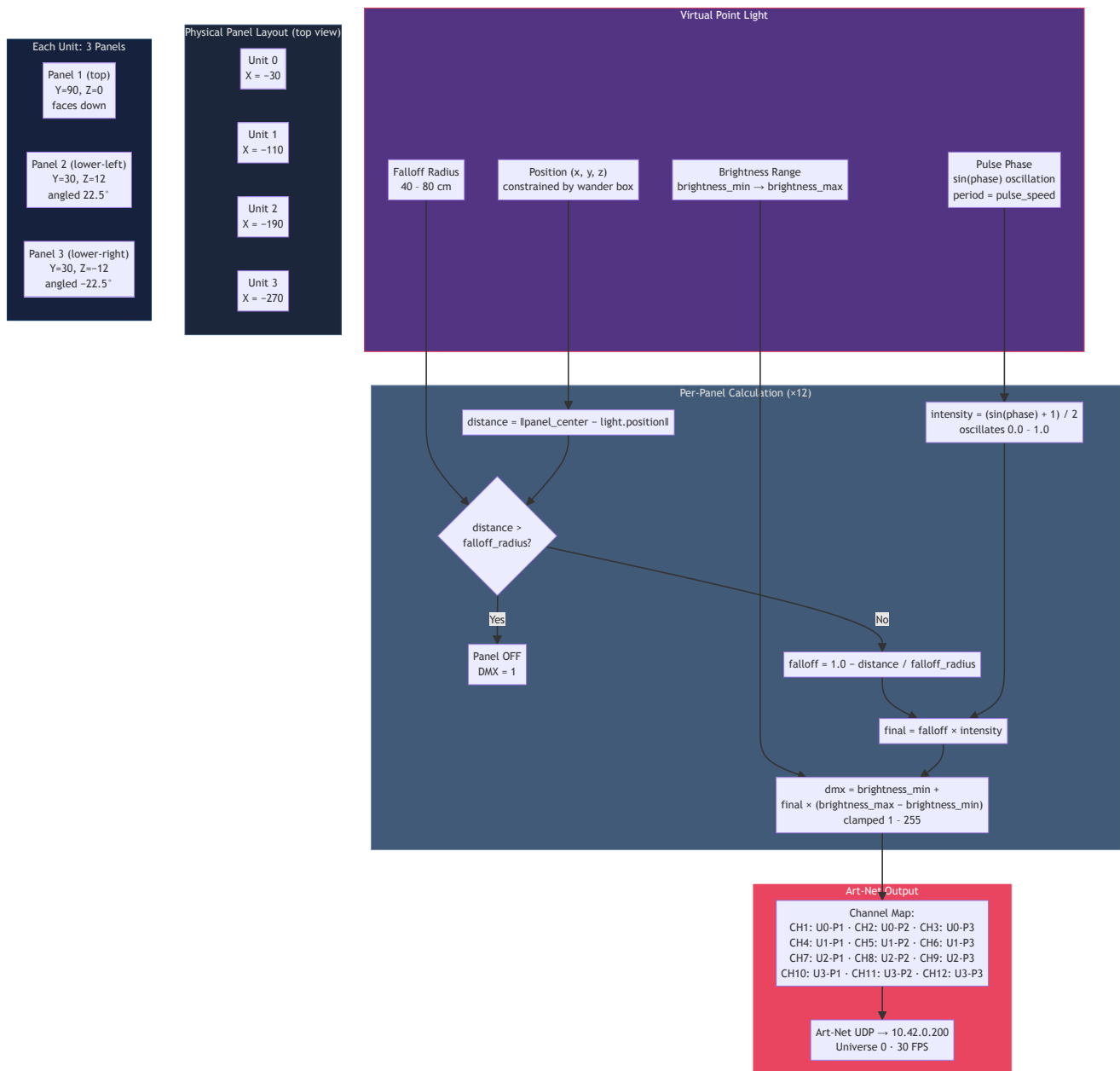
Buffer: 50 recent engagement contexts
Dims: position x aggression x time x flow
Rate: +/-0.02 per engagement

How the timescales connect:

- The **minute-scale** trend windows feed the **5-second** auto-tuning cycle as activity signals
- The **daily** learning cycle sets starting home values for the next day's auto-tuning
- The **weekly** engagement history informs daily learning (7-day weighted average)
- **Per-frame** rendering reads the MetaParameters that the 5-second tuner has adjusted
- **Flow tracking** (1.5s) feeds both the per-frame flow mode logic and the minute-scale trend analysis

7. Light Position → Panel DMX Output

The final stage converts the virtual point light into 12 physical DMX values. The `PanelSystem` models the exact physical layout of the installation: 4 lighting units mounted at different X positions, each containing 3 LED panels at different angles.



The brightness equation in plain language:

1. Measure the distance from the light to each panel's center point
2. If the panel is outside the falloff radius, it's off (DMX = 1)
3. Otherwise, compute a falloff factor: closer panels are brighter (linear decay)
4. Multiply the falloff by the current pulse intensity (a sine wave that breathes between 0 and 1)
5. Map the result into the brightness range and convert to a DMX byte (1-255)

The falloff radius is the single most impactful parameter on how the light "looks." A small radius (40cm in CROWD mode) creates a tight spotlight that isolates individual panels. A large radius (80cm in IDLE mode) creates a soft wash across multiple panels.

Wander Box: Behavior Inputs to Spatial Motion

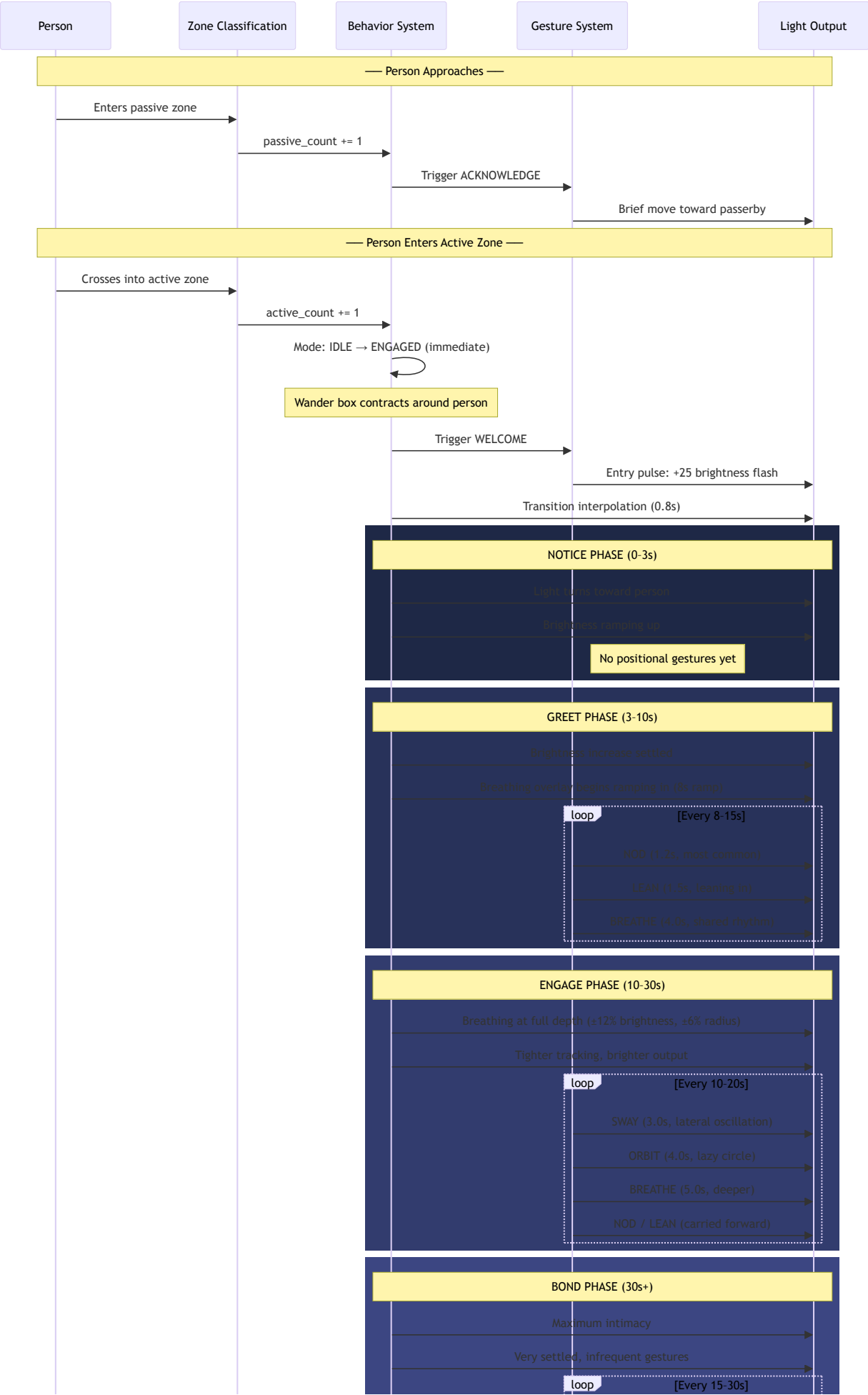
The wander box is a 3D bounding volume that constrains where the virtual point light can move. Rather than roaming freely, the light picks random targets inside this box and lerps toward them, creating the characteristic slow drift seen in IDLE mode and the tight tracking seen in ENGAGED mode. Every behavior modifier — flow direction, aggression, engagement contraction — works by reshaping the box, not by steering the light directly.

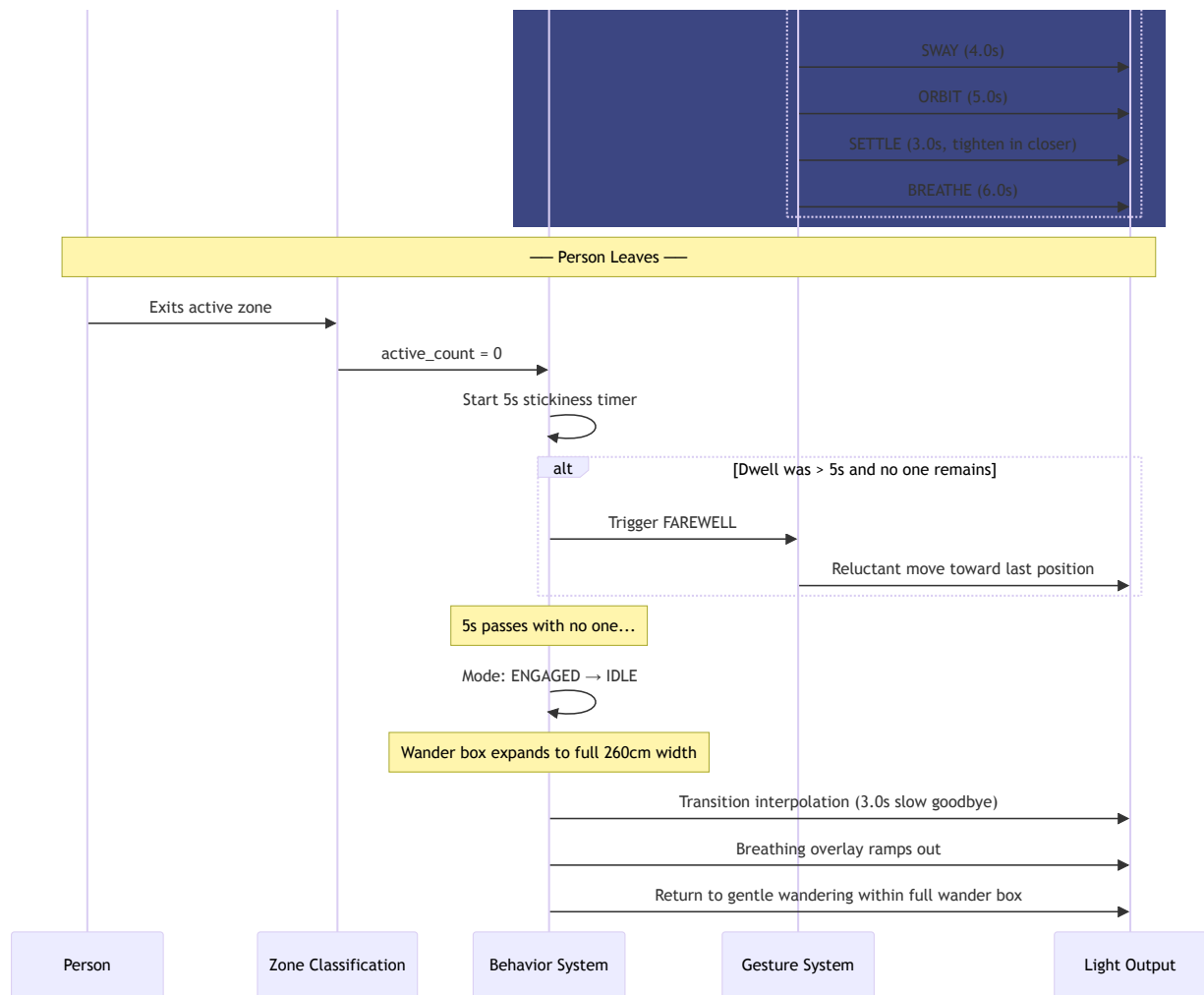
The box uses a three-layer animation system: `current_wander_box` reflects the raw mode and modifier state, `animated_wander_box` smooths that with an exponential lerp (speed 3.0, ~95% converged in one second), and `WanderBehavior` picks random points inside the animated box at timed intervals. This layered approach prevents jarring jumps when mode or engagement state changes.



8. Engagement Lifecycle

This diagram traces a complete person interaction from arrival to departure, showing which systems activate at each stage. The dwell phases progressively deepen the connection, while the gesture system and breathing overlay add expressive texture throughout.





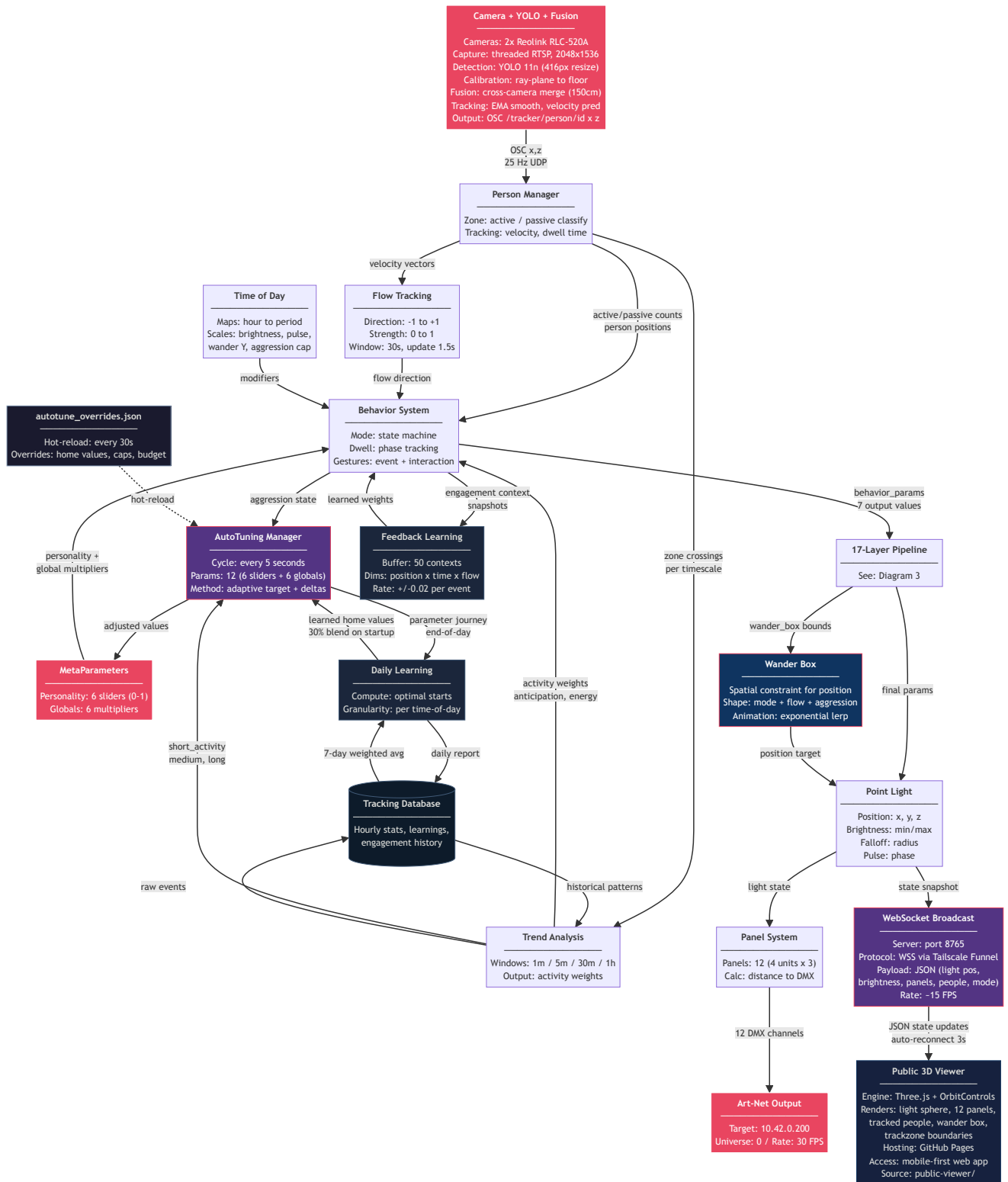
What the person experiences:

1. Walking past, the light briefly acknowledges them — a subtle flicker of awareness
2. Stepping under the panels, the light immediately locks on with a welcoming pulse
3. Standing still, they notice the light beginning to breathe — a slow shared rhythm
4. After 10 seconds, the light starts to sway and orbit gently, as if comfortable in their presence
5. After 30 seconds, the light settles in close — maximum intimacy, minimal movement
6. Walking away, the light lingers, reluctantly following their last position before slowly fading back to its wandering state

The entire interaction is shaped by the current MetaParameters — a highly social, energetic personality will greet faster, gesture more frequently, and track more tightly. A shy, calm personality will be subtle and slow, with longer pauses between gestures.

Appendix: How Everything Connects

This final diagram shows the complete system with all feedback loops visible at once — the "full picture" view of how inputs, processing layers, adaptation loops, and outputs relate to each other.



Part 2: Camera Tracking System

The following diagrams explain how two RTSP cameras capture video, detect people using YOLO, project detections to real-world floor coordinates via calibration, fuse cross-camera observations, and output tracked positions over OSC. This is the upstream data source that feeds the behavior system described in Part 1.

Diagram 9 — Camera System Overview

High-level pipeline from physical cameras to the OSC messages consumed by the light controller. Two Reolink PoE cameras feed RTSP streams into threaded capture buffers, YOLO detects people, calibration projects pixel positions onto the floor plane, and cross-camera fusion produces stable world-coordinate tracks. The final OSC messages carry person count and per-person (id, x, z) positions to `lightController_osc.py` at roughly 25 updates per second.

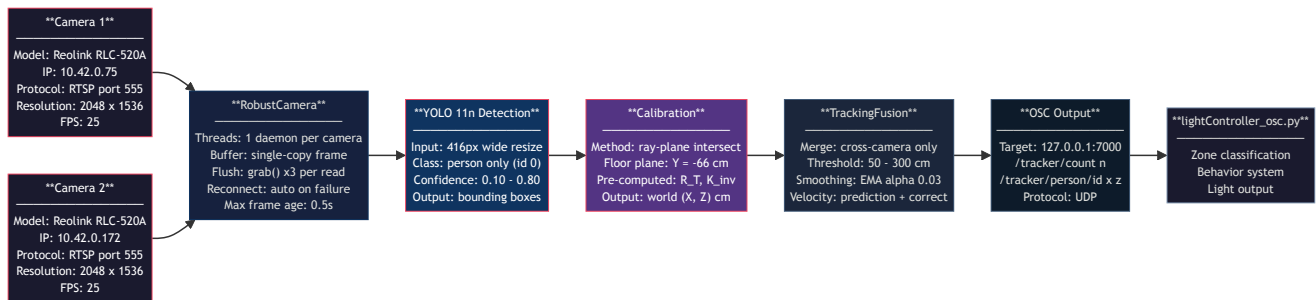
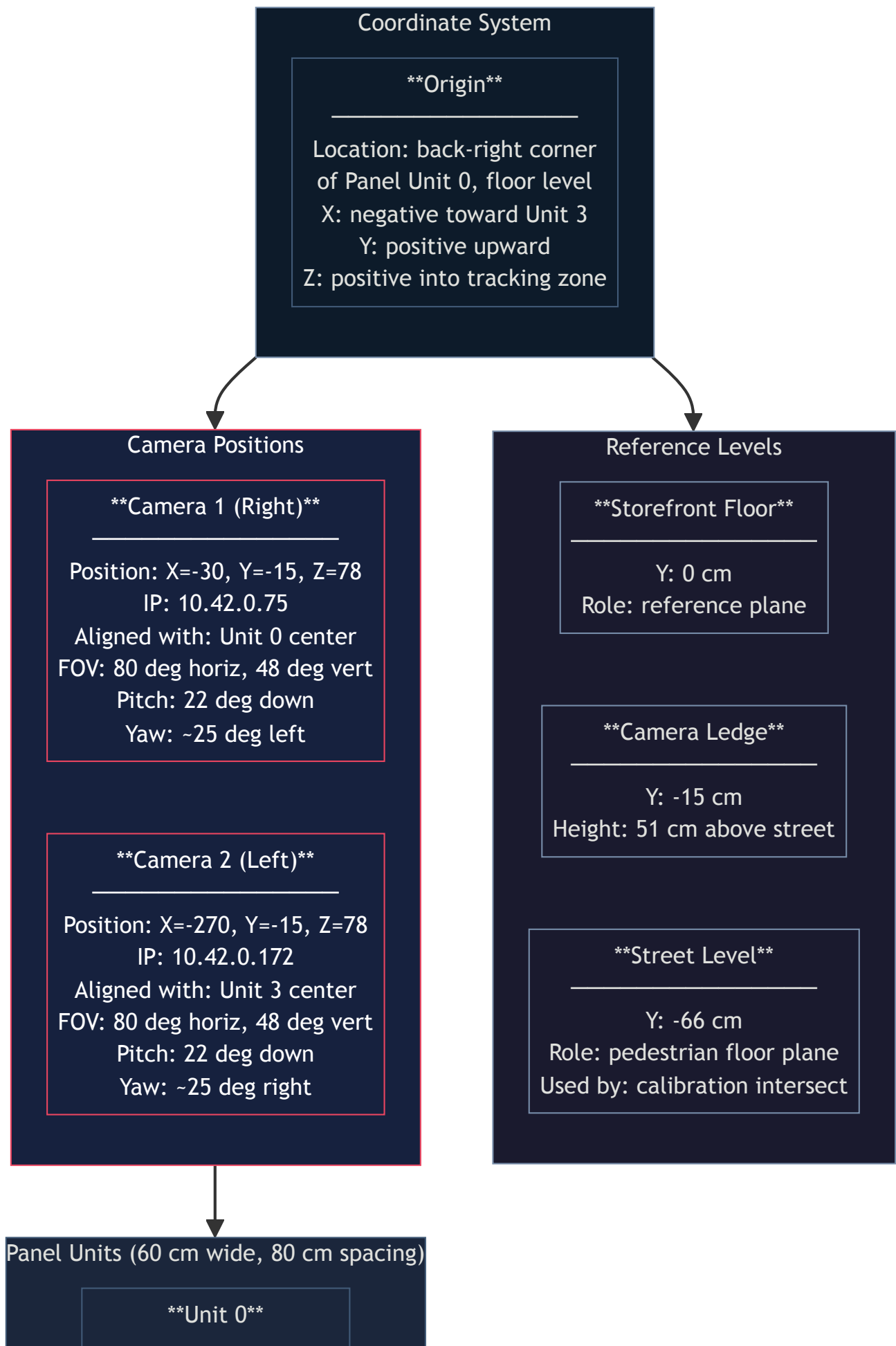


Diagram 10 — Physical Setup and Coordinate System

Camera positions, ArUco calibration markers, and tracking zones placed in the real-world coordinate system. The origin sits at the back-right corner of Panel Unit 0 at floor level, with X running negative toward Unit 3, Y pointing up, and Z extending outward into the sidewalk. Seven ArUco markers at known 3D positions provide the calibration reference, and two depth zones (active and passive) define the engagement boundaries used by the behavior system.



Center X: -30
Edges: 0 to -60

****Unit 1****

Center X: -110
Edges: -80 to -140

****Unit 2****

Center X: -190
Edges: -160 to -220

****Unit 3****

Center X: -270
Edges: -240 to -300



Tracking Zones

****Active Zone****

X: -350 to 50
Z: 78 to 283
Depth: ~2 m from panels
Role: engaged interaction

****Passive Zone****

X: -350 to 50
Z: 283 to 553

Depth: ~2.7 m beyond active
Role: sidewalk passersby

Diagram 11 — Camera Capture and YOLO Detection

Each camera runs a dedicated daemon thread that continuously reads RTSP frames and stores the most recent one under a lock. The main tracking loop retrieves these frames, resizes them to 416 pixels wide for speed, and passes them through YOLO 11n to detect people. Detected bounding boxes are scaled back to full resolution and cached per camera so that unchanged frames reuse the previous detection without re-running inference.

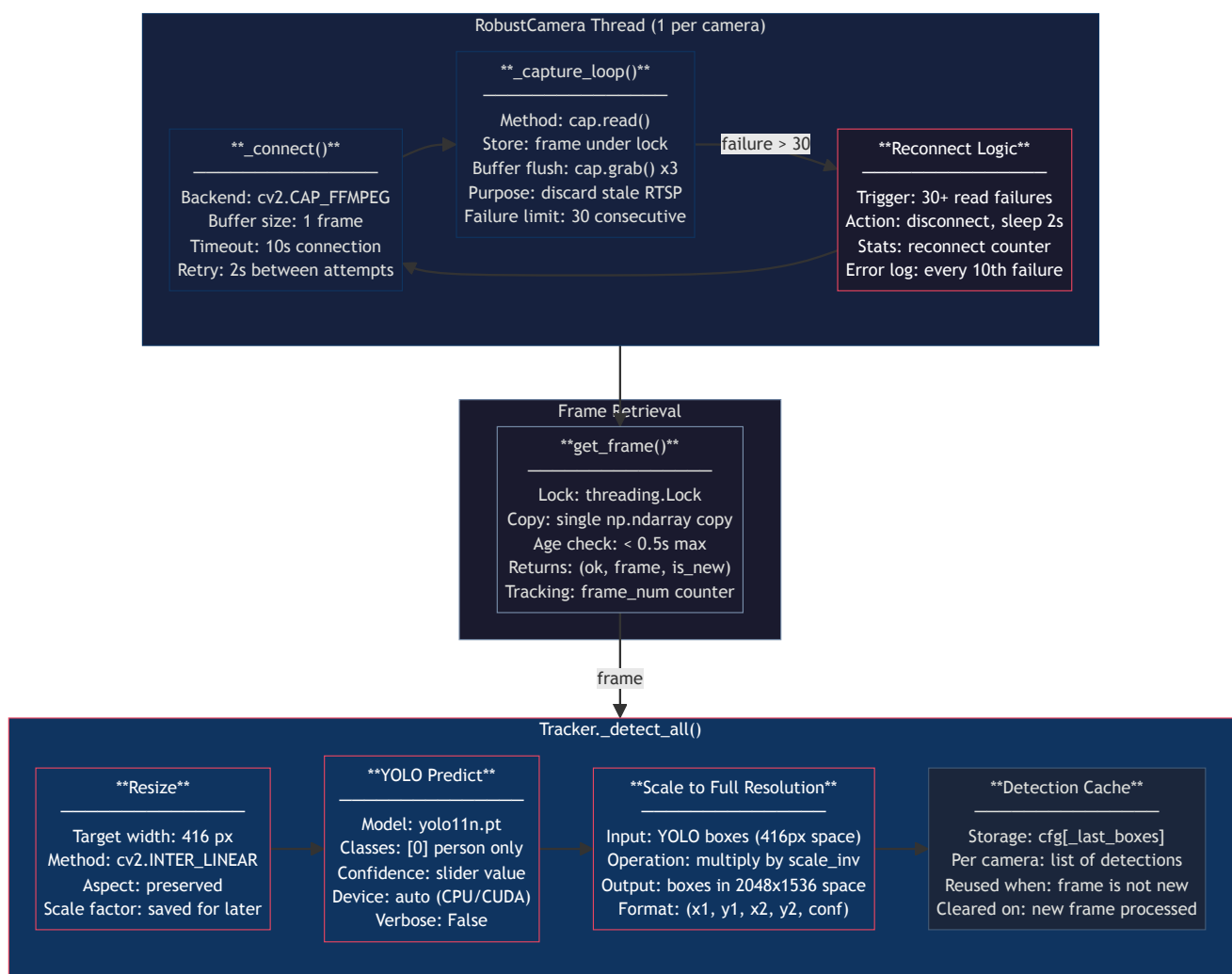
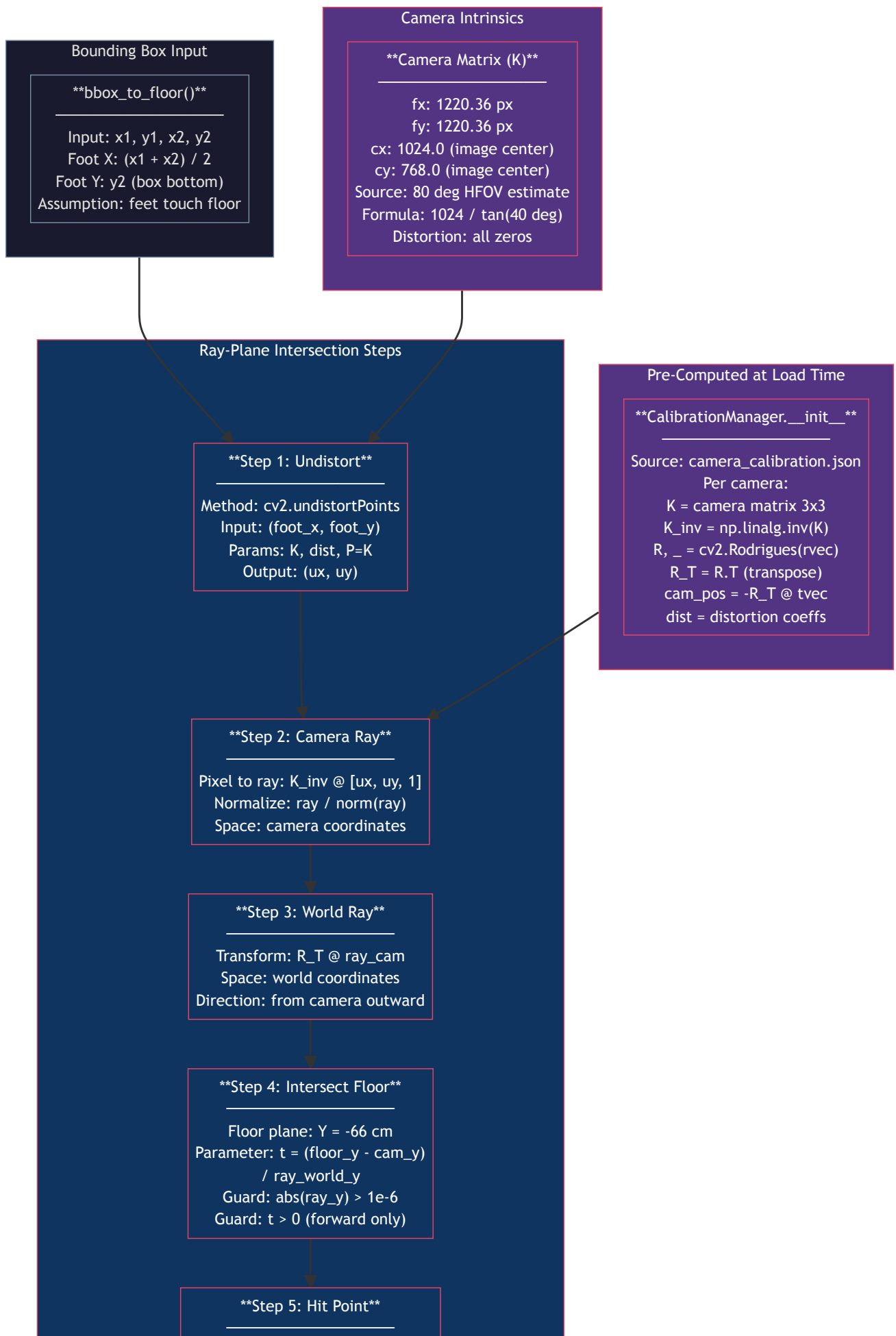


Diagram 12 — Calibration: Pixel to World Coordinates

The ray-plane intersection math that transforms a bounding-box foot position in image pixels to a real-world floor position in centimeters. For each detection, the bottom-center of the bounding

box is treated as the person's feet; that pixel is undistorted, projected into a camera-space ray using the inverse intrinsic matrix, rotated into world space, and intersected with the known floor plane at $Y = -66$ cm. The result is a (world_x, world_z) pair in centimeters that feeds directly into the fusion stage.



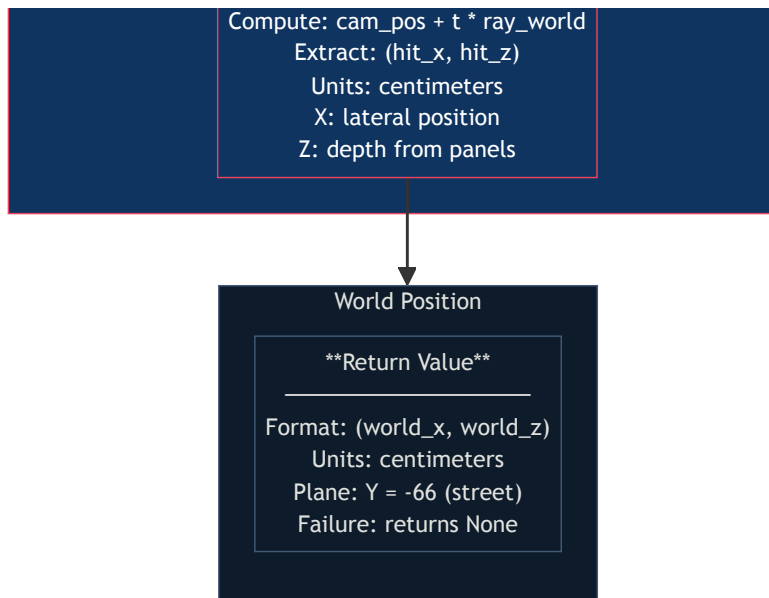


Diagram 13 — Cross-Camera Fusion and Temporal Tracking

Detections from the two cameras are fused spatially and then matched temporally to produce stable, smoothed tracks. Spatial fusion uses greedy nearest-neighbor matching across cameras only (same-camera detections are never merged) with a configurable distance threshold. Temporal tracking predicts each existing track's next position using its velocity, matches incoming detections to predictions, applies exponential moving average smoothing, and prunes tracks unseen for more than 60 frames.

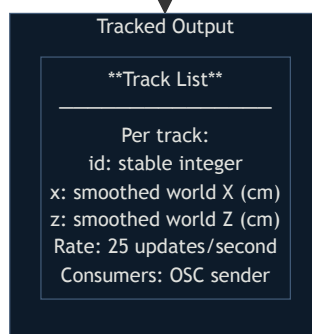
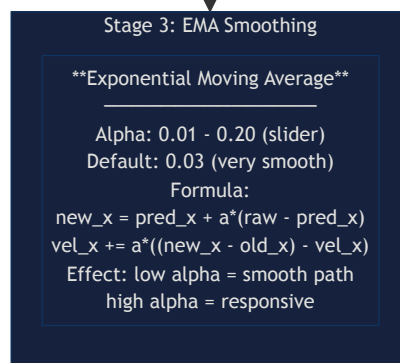
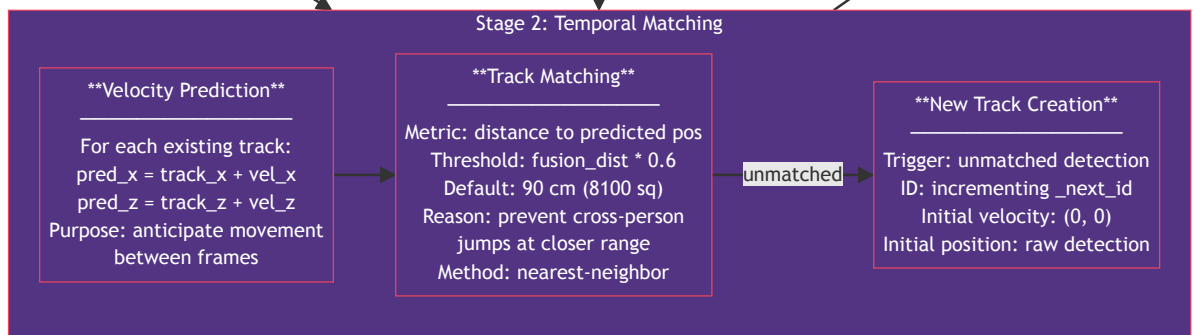
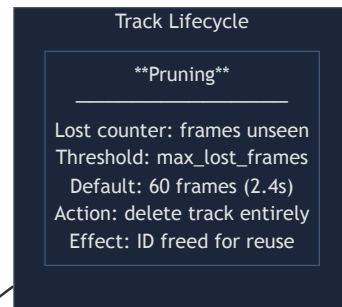
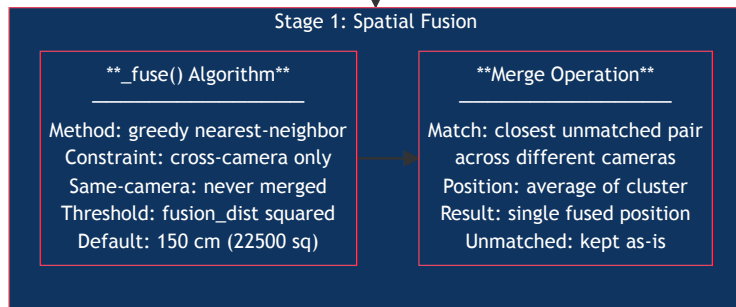
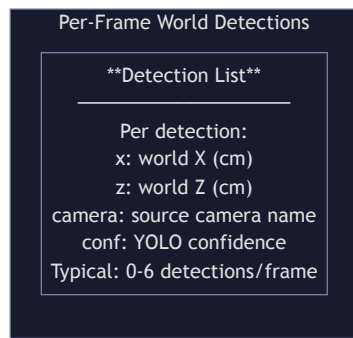
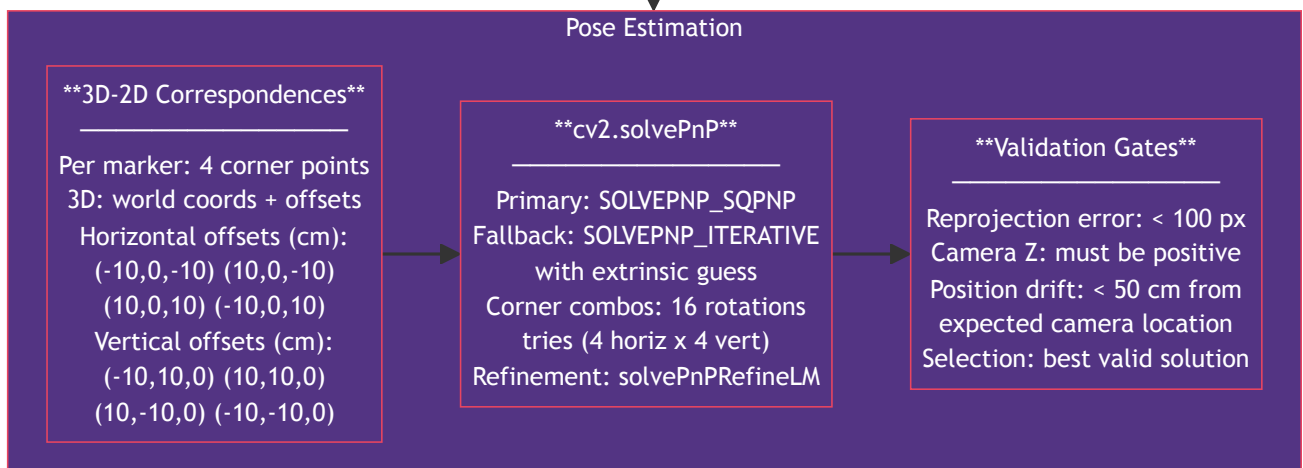
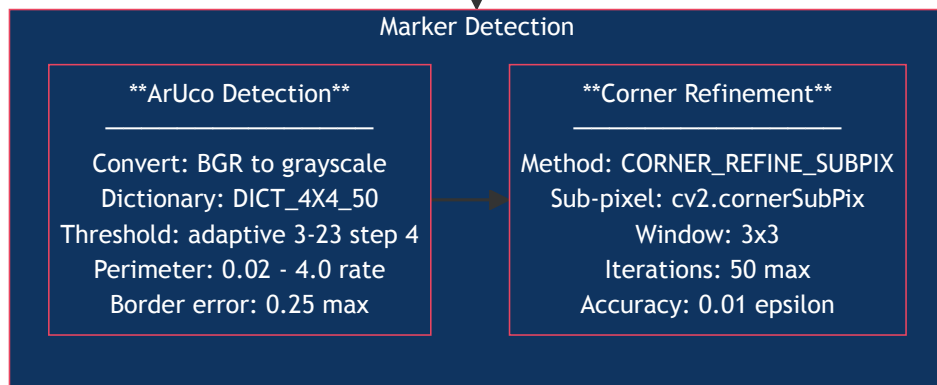
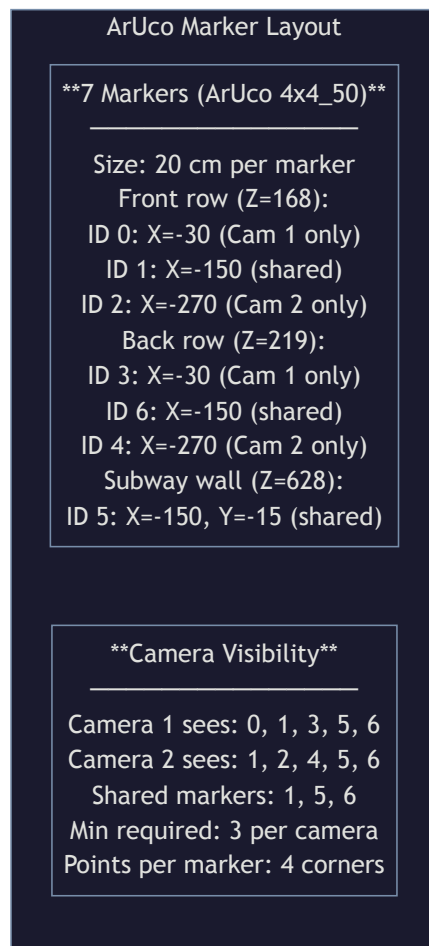


Diagram 14 — ArUco Calibration Process

The one-time camera calibration uses seven ArUco markers placed at known 3D positions to compute each camera's rotation and translation via `cv2.solvePnP`. Each marker provides four corner correspondences between known world coordinates and detected image pixels, giving enough constraints for a robust pose estimate. The solver tries multiple corner orderings and validates results against reprojection error, camera position sanity, and expected location drift before saving the best solution to `camera_calibration.json`.



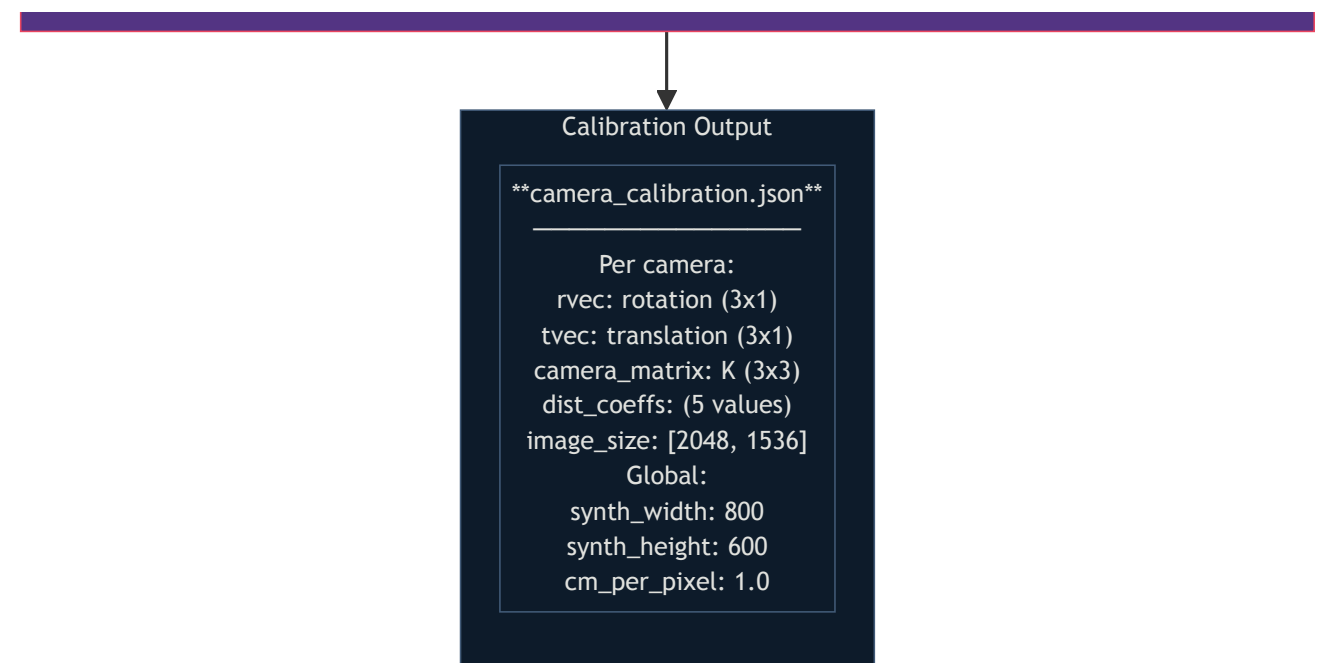


Diagram 15 — Main Loop and Tunable Parameters

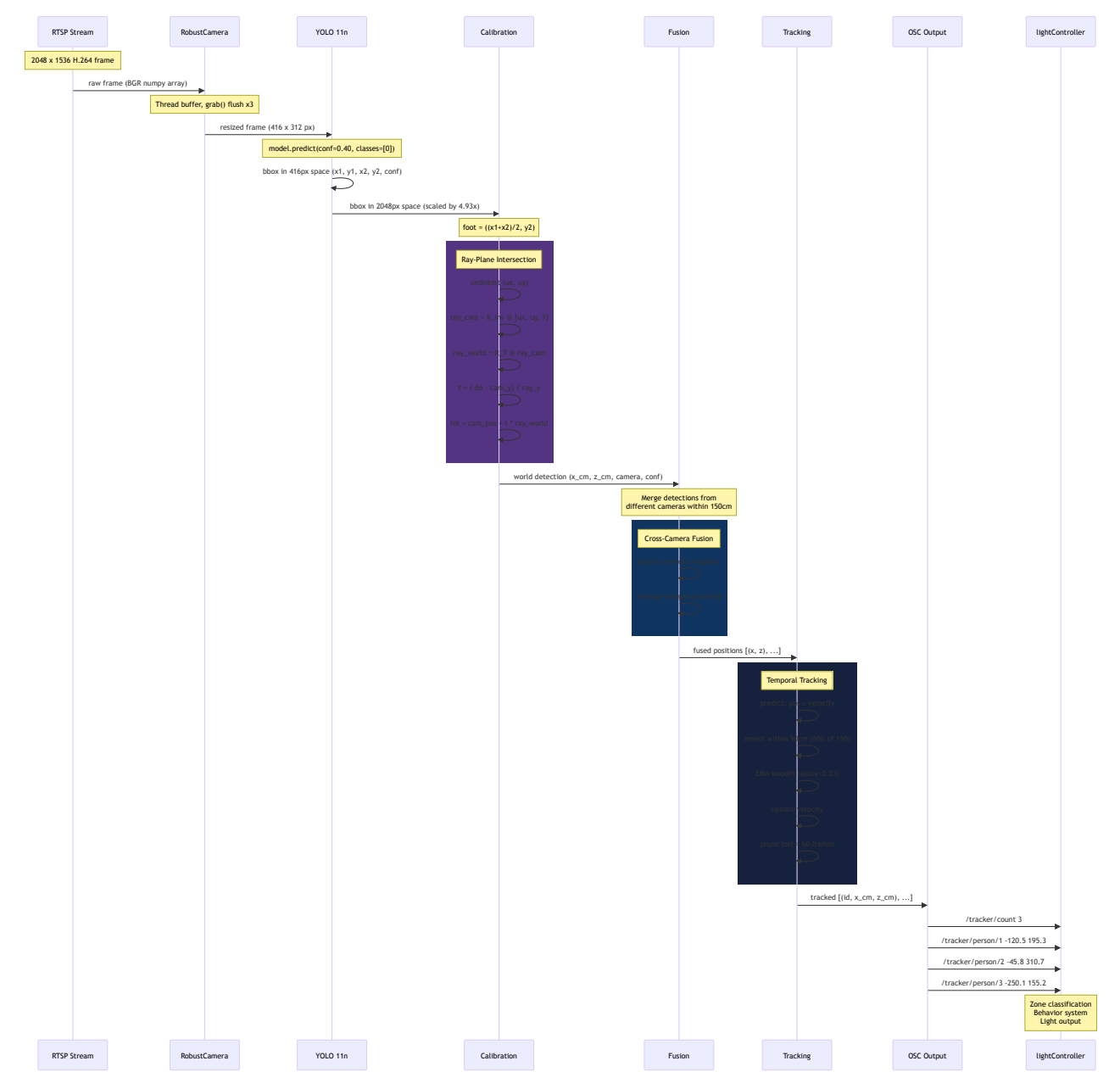
The tracker's main loop runs at a fixed 25 FPS, cycling through detection, fusion, OSC output, and optional visualization on every frame. Three live slider parameters — confidence threshold, fusion distance, and EMA smoothing alpha — let the operator tune the tradeoff between responsiveness and stability without restarting. Periodic background tasks auto-save settings every 5 seconds, log health metrics every 5 minutes, and reload the YOLO model hourly to prevent tracker drift.



Diagram 16 — End-to-End Data Transform

A sequential walkthrough showing the exact data shape at each stage of the pipeline, from a 2048x1536 H.264 frame to the final OSC messages consumed by the behavior system. Each

handoff is annotated with the concrete values and formats involved — pixel coordinates, scale factors, ray equations, fusion thresholds, and EMA parameters. This diagram serves as a debugging reference: if tracking behaves unexpectedly, you can trace the data transforms step by step to isolate where the problem occurs.



Cross-References

Resource	Description
BEHAVIOR_SYSTEM.md	Full prose reference for all behavior systems
light_behavior.py	State machine, gestures, trend analysis, feedback learning
lightController_osc.py	Main loop, OSC, Art-Net, WebSocket, AutoTuning

Resource	Description
public-viewer/	Three.js web viewer (connects via WebSocket)
V2_5Dev/camera_tracker_osc.py	Camera tracker V2.5 (RTSP, YOLO, calibration, fusion, OSC)
camera_calibration.py	Camera calibration tool (ArUco, solvePnP, synth view)
camera_calibration.json	Calibration data (rvec, tvec, K, distortion per camera)
world_coordinates.json	ArUco marker 3D positions and camera intrinsics

These diagrams describe the behavior and camera tracking systems as of the current codebase. The auto-tuning, feedback learning, and daily learning mechanisms mean the light's personality evolves over time — these diagrams capture the architecture, but the actual parameter values will drift as the system adapts to its environment.